

UNIVERSIDAD RAFAEL LANDÍVAR
INGENIERÍA EN INFORMÁTICA Y SISTEMAS
ING. MANUEL ROJAS
ESTRUCTURA DE DATOS II
4TO SEMESTRE
PROYECTO 2 DE ESTRUCTURA DE DATOS
APLICACIÓN DE ÁRBOL B+, MIN HEAP Y MAX HEAP
TABLA DE POSICIONES DE JUGADORES DEL MUNDIAL
MANUAL DE USUARIO
PABLO DAVID QUIJIVIX SUCHI 1578125



Universidad
Rafael Landívar
Tradición Jesuita en Guatemala

Manual de Usuario

Sistema de Tabla de Posiciones de Jugadores del Mundial

Proyecto en C# — Árbol B+, Min Heap y Max Heap

1. Descripción general

Este programa es un sistema de consola desarrollado en C# que permite gestionar la información de los jugadores que participan en el Mundial: sus datos personales (nombre, selección, posición) y sus estadísticas de desempeño (minutos jugados, goles, asistencias, tarjetas amarillas, tarjetas rojas y partidos disputados).

El sistema permite:

- Registrar nuevos jugadores manualmente desde el teclado.
- Buscar rápidamente los datos de un jugador específico.
- Eliminar un jugador del sistema.
- Ver el listado completo de todos los jugadores registrados.
- Calcular el Top 5 de jugadores según una categoría (goles, asistencias, minutos jugados, tarjetas amarillas, tarjetas rojas o partidos disputados).
- Generar un listado general ordenado (de mayor a menor) por la categoría que el usuario elija.
- Cargar automáticamente todos los jugadores guardados previamente cada vez que se abre el programa, y guardar automáticamente cualquier cambio (persistencia en tiempo real).

Internamente, el sistema NO usa colecciones nativas de C# ('List', 'Dictionary', 'SortedDictionary', 'SortedSet', 'PriorityQueue') para su lógica principal. En su lugar, implementa desde cero:

- Un Árbol B+ — estructura principal donde se almacenan todos los jugadores, indexados por su Id. Permite insertar, buscar y eliminar de forma eficiente, y recorrer todos los registros ya ordenados.
- Un Min Heap — usado para calcular el Top 5 de cada categoría de forma eficiente.
- Un Max Heap — usado para generar el listado general ordenado de mayor a menor por la categoría elegida.

2. Requisitos previos

Para ejecutar el programa necesita tener instalado:

Cualquier IDE con .NET, pero no se necesita Visual Studio o otro programa adicional: todo se ejecuta

desde la terminal con el SDK de .NET.

3. Contenido del proyecto (estructura de carpetas) MundialJugadores

- MundialJugadores.csproj (archivo de configuración del proyecto)
- Program.cs (punto de entrada del programa)

Modelos

- Jugador.cs (clase que representa a un jugador)

Estructuras

- BPlusTree
 - NodoB.cs (nodo del Árbol B+)
 - ArbolBMas.cs (lógica completa del Árbol B+)

Heaps

- MinHeap.cs (montículo mínimo, para el Top 5)
- MaxHeap.cs (montículo máximo, para el listado ordenado)

Módulos

- ModuloInsertar.cs (registrar jugador desde consola)
- ModuloBuscar.cs (buscar jugador por Id)
- ModuloEliminar.cs (eliminar jugador por Id)
- ModuloRecorrer.cs (ver listado completo, ordenado por Id)
- ModuloImprimir.cs (Top 5 y listado ordenado por categoría)
- ModuloCargaCSV.cs (cargar jugadores desde el archivo .csv)

- ModuloCargaTXT.cs (cargar jugadores desde el archivo .txt)

Persistencia

- GestorPersistencia.cs (guarda los cambios en los archivos .csv/.txt)

Menú

- MenuPrincipal.cs (menú interactivo de consola)

Datos

- jugadores.csv (base de datos principal, formato CSV)
- jugadores.txt (base de datos secundaria, formato TXT)

4. Cómo ejecutar el programa (paso a paso)

1. Abre una terminal:

- En Windows: busca "cmd" o "PowerShell" en el menú de inicio.
- En macOS/Linux: abre la aplicación de Terminal.
- O abre algún IDES.
- Se debe de tener si o si el paquete de “.NET” instalado.

2. Muevase hasta la carpeta del proyecto con el comando `cd`, por ejemplo:

'''

cd DondeGuardeeldelarchivo/MundialJugadores

'''

3. Verifique que esté en la carpeta correcta escribiendo:

'''

dir (en Windows)

ls (en macOS/Linux)

'''

Debe ver el archivo `MundialJugadores.csproj` en el listado.

4. Ejecutar el programa con:

'''

dotnet run

'''

5. La primera vez que lo ejecute, `dotnet` compilará el proyecto automáticamente (puede tardar unos segundos). Luego verá un mensaje como:

'''

Cargando datos persistidos...

Se cargaron 12 registro(s) desde el .csv y 3 desde el .txt (los Id repetidos solo se actualizan, no se duplican).

'''

Y a continuación aparecerá el menú principal.

Alternativa: compilar y luego ejecutar por separado

'''

dotnet build

dotnet bin/Debug/net8.0/MundialJugadores.dll

'''

6. El menú principal

Al iniciar el programa, verá lo siguiente:

'''

SISTEMA DE JUGADORES DEL MUNDIAL

1. Insertar nuevo jugador

2. Buscar jugador por Id

3. Eliminar jugador

4. Recorrer todos los jugadores (Árbol B+, ordenado por Id)

5. Top 5 por categoría (Min Heap)

6. Listado general ordenado por categoría (Max Heap)

0. Salir

Seleccione una opción:

'''

Se debe escribir el número de la opción deseada y presionar “Enter”. A continuación se explica cada opción en detalle.

Opción 1 — Insertar nuevo jugador

Debe ingresar, uno por uno, los siguientes datos:

Campo	Tipo	Ejemplo
-------	------	---------

---	---	---
-----	-----	-----

Id	Texto único	J016
----	-------------	------

Nombre	Texto	Kevin Alvarado
--------	-------	----------------

Selección	Texto	Guatemala
-----------	-------	-----------

Posición	Texto	Delantero
----------	-------	-----------

Minutos jugados	Número entero ≥ 0	450
Goles	Número entero ≥ 0	3
Asistencias	Número entero ≥ 0	2
Tarjetas amarillas	Número entero ≥ 0	1
Tarjetas rojas	Número entero ≥ 0	0
Partidos disputados	Número entero ≥ 0	5

Reglas y validaciones:

- El Id no puede estar vacío ni repetirse. Si ya existe un jugador con ese Id, el programa lo indicará y cancelará el registro.
- Para actualizar datos de un jugador existente, primero elimínalo con la opción 3 y vuelve a insertarlo, o edita directamente el archivo `.csv`/`.txt`).
- Todos los campos numéricos deben ser números enteros mayores o iguales a 0. Si escribe texto o un número negativo, el programa lo pedirá de nuevo hasta que ingrese un valor válido.
- Ningún campo de texto puede quedar vacío.

Al finalizar, el jugador queda guardado inmediatamente tanto en el Árbol B+ (en memoria) como en los archivos `jugadores.csv` y `jugadores.txt` (en disco).

Opción 2 — Buscar jugador por Id

Se pedirá el Id del jugador (por ejemplo `J001`) y mostrará todos sus datos si existe, o un mensaje indicando que no se encontró, en caso contrario.

Ejemplo de salida:

```

#### Jugador encontrado:

[J001] Lucas Martinez      | Argentina      | Delantero      | Min: 540 | Gol: 5 | Ast: 2 | TA: 1  
| TR: 0 | PJ: 6  
```

Opción 3 — Eliminar jugador

Se pedirá el Id del jugador a eliminar. Si el Id existe, el jugador se elimina del árbol y los archivos `.csv`/`.txt` se actualizan de inmediato. Si el Id no existe, se muestra un aviso y no se realiza ningún cambio.

Opción 4 — Recorrer todos los jugadores

Muestra el listado completo de todos los jugadores registrados, ordenados alfabéticamente/numéricamente por su Id (este es el recorrido natural del Árbol B+, recorriendo sus hojas de izquierda a derecha).

Opción 5 — Top 5 por categoría (Min Heap)

Primero pedirá elegir una categoría, escribiendo exactamente una de estas palabras:

```
'''
goles
asistencias
minutos
tarjetasamarillas
tarjetasrojas
partidos
'''
```

Si escribe algo distinto, el programa lo volverá a pedir. Luego mostrará los 5 jugadores con el valor más alto en esa categoría, ordenados de mayor a menor, junto con el valor exacto de la categoría elegida.

Ejemplo (categoría "goles"):

```
'''
--- TOP 5 por 'goles' (Min Heap) ---
1. [J008] Carlos Silva ... -> goles: 7
2. [J006] Diego Fernandez ... -> goles: 6
3. [J001] Lucas Martinez ... -> goles: 5
4. [J011] Nicolas Duval ... -> goles: 5
5. [J003] Kenji Tanaka ... -> goles: 4
'''
```

(Los valores reales dependerán de los datos que tengas cargados en ese momento).

Opción 6 — Listado general ordenado por categoría (Max Heap)

Igual que la opción 5, primero pide elegir una categoría, pero en vez de mostrar solo el Top 5, muestra a todos los jugadores registrados, ordenados de mayor a menor según esa categoría. Es útil para ver la clasificación

completa (por ejemplo, la tabla completa de goleadores).

Opción 0 — Salir

Cierra el programa. No es necesario "guardar" antes de salir: como se explica en la siguiente sección, los archivos ya están actualizados en todo momento.

6. Persistencia de datos (cómo se guardan los cambios)

El sistema mantiene siempre sincronizados dos archivos dentro de la carpeta `Datos/`:

- jugadores.csv — formato de valores separados por comas, con encabezado.
Ejemplo de una línea:

J001,Lucas Martinez,Argentina,Delantero,540,5,2,1,0,6

-jugadores.txt — formato de texto separado por el carácter `|`, sin encabezado. Ejemplo de una línea:

J001|Lucas Martinez|Argentina|Delantero|540|5|2|1|0|6

En ambos casos, el orden de los campos es siempre:

```

Id, Nombre, Selección, Posición, MinutosJugados, Goles, Asistencias,  
TarjetasAmarillas, TarjetasRojas, PartidosJugados

```

¿Cuándo se actualizan estos archivos?

- Automáticamente al iniciar el programa (para normalizar el estado inicial).
- Automáticamente cada vez que insertas un jugador nuevo (Opción 1).
- Automáticamente cada vez que eliminas un jugador (Opción 3).

No se necesita guardar manualmente ni cerrar el programa para que los cambios se reflejen en disco: cada operación que modifica datos reescribe ambos archivos

por completo con el estado actual del sistema.

¿Puede editar los archivos? csv`/`.txt` directamente con Excel/Bloc de notas?

Sí. Si se agrega o se modifica líneas manualmente respetando el formato de columnas indicado arriba, el programa las cargará la próxima vez que se inicie. Se debe tener cuidado de no dejar campos numéricos vacíos o con texto, ya que esas líneas se ignorarán silenciosamente al cargar.

7. Preguntas frecuentes / solución de problemas

"dotnet no se reconoce como un comando"

No se tiene instalado el SDK de .NET, o no está agregado al PATH del sistema. Instalación desde `https://dotnet.microsoft.com/download` y reinicia la terminal.

El programa dice "Se cargaron 0 registros"

Verifica que los archivos `Datos/jugadores.csv`` y `Datos/jugadores.txt`` existan y no estén vacíos, y que se ejecuto `dotnet run`` desde dentro de la carpeta `MundialJugadores`` (donde está el `.csproj``), no desde una carpeta distinta.

Quiero empezar desde cero, sin los jugadores de ejemplo

Abre `Datos/jugadores.csv`` y deja solo la primera línea (el encabezado); y vacía por completo `Datos/jugadores.txt``. La próxima vez que ejecutes el programa, no se cargará ningún jugador y podrás empezar a registrar los tuyos.

Un campo numérico no acepta mi valor

Los campos numéricos (minutos, goles, asistencias, tarjetas, partidos) solo aceptan números enteros mayores o iguales a 0. No se permiten decimales, letras ni números negativos.

¿Hay un límite de jugadores que puedo registrar?

No hay un límite fijo en el código: el Árbol B+ y los Heaps crecen dinámicamente (duplicando su capacidad interna cuando hace falta), por lo que puedes registrar tantos jugadores como la memoria de tu computadora permita.

8. Resumen técnico (para la sustentación/documentación del proyecto)

Estructura: Árbol B+

Uso en el sistema: Almacenamiento principal de todos los jugadores, indexados por Id. Soporta Insertar, Buscar, Eliminar y Recorrer.

Justificación:

Las operaciones más frecuentes del sistema son buscar un jugador puntual e insertar/actualizar durante el torneo; el B+ resuelve ambas en $O(\log n)$ y, gracias al enlace entre hojas, también entrega el recorrido ordenado completo en $O(n)$ sin recursión, cubriendo el requisito de "listado general".

Estructura: Min Heap

Uso en el sistema: Cálculo del Top 5 de cada categoría estadística.

Justificación: Mantener un heap de tamaño fijo 5 permite encontrar el Top 5 en $O(n \log 5)$, sin necesidad de ordenar todos los jugadores.

Estructura: Max Heap

Uso en el sistema: Generación del listado completo ordenado de mayor a menor por la categoría elegida.

Justificación: Insertar todos los jugadores y extraer el máximo repetidamente produce el orden descendente completo en $O(n \log n)$, sin usar un método de ordenamiento nativo.